

DATE : 05.10.2026
DT/NT : DT
LESSON : LINUX
SUBJECT: BASH SCRIPTING PROJECT

BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



Table of Contents

- ▶ Project Overview & Bash Rules
- ▶ System Information & Variables
- ▶ File & Directory Inspection
- ▶ Resource, Process & Network Checks
- ▶ User Input, Arguments & Decision Logic
- ▶ Administration Menu & Reports
- ▶ Final Linux Administration Toolkit

Project objective: turn manual Linux administration commands into repeatable, readable Bash tools.





Allowed Building Blocks

A deliberately limited Bash feature set keeps the project focused on fundamentals.

You may use

- echo, variables, command substitution
- read and positional argument \$1
- if / elif / else
- for loops and case
- Linux commands: ls, whoami, hostname, date, uname, id, ps, ip, df, free, uptime, grep, find
- Output redirection: > and >>

Do not use

- Arrays
- Functions
- Bash file tests: -f, -d, -r, -w, -x
- Debugging tools

System Administration Project

- Bash automation for junior Linux System Administrators



System Administration Scenario



Scenario

You are working as a junior Linux System Administrator. You already know how to manage files, directories, permissions, processes, storage, memory, users, and networking manually from the terminal.

Manager request

Automate common administrative checks using Bash scripts that collect information, inspect files and directories, check permissions, monitor resources, inspect processes and networking, interact with the administrator, and generate simple reports.

System Administration Roadmap

14 guided parts lead into one final Linux Administration Toolkit.



Parts 1-2 — System Information + Variables



Part 1 — system-info.sh

Display:

- Current User
- Hostname
- Date
- Kernel
- User ID and Groups
- Uptime

Evidence: run `./system-info.sh` and show the result.

Part 2 — Store results in variables

Modify the same script so at least three command results are stored in variables before being printed.

Focus: What is the advantage of storing command output inside a variable?

```
current_user=$(whoami)
host=$(hostname)
now=$(date)

echo "$current_user"
```

Parts 3–5 — File and Directory Inspection



Part 3 — file-audit.sh

Inspect project areas and important files. Show permissions, ownership, and readable sections for public, private, developers, and shared.

Part 4 — directory-check.sh

Use a for loop to inspect public, private, developers, and shared automatically.

Do not use an array.

Part 5 — Important files loop

Use another for loop for announcement.txt, secrets.txt, project.txt, and deploy.sh to avoid repeating the same command.

Parts 6–8 — Resources, Processes, Networking



Part 6 — resource-check.sh

Show uptime, memory usage, and disk usage.

Commands: uptime, free, df

Discuss why administrators monitor free disk space.

Part 7 — process-check.sh

Show running processes and current user.

Challenge: start sleep 300 & and locate it.

Discuss PID and background execution.

Part 8 — network-info.sh

Show hostname, interfaces, IP addresses, and routing information.

Parts 9–10 — User Input and Script Arguments



Part 9 — inspect-path.sh

Ask: “Enter a file or directory to inspect:”
Use `read`, then display information about the entered path.

Test with `public`, `private`, `developers`, `shared`, and at least one file.

Part 10 — quick-check.sh

Pass the path when launching the script instead of asking with `read`.

Use `$1` as the first command-line argument.

```
./quick-check.sh developers
./quick-check.sh private/secrets.txt

target=$1
```

Parts 11–12 — Decision Logic + Administration Menu



Part 11 — admin-check.sh

Use if / elif / else to respond differently when the user selects public, private, developers, or shared. Then display the relevant directory information.

Part 12 — admin-tool.sh

Create an 8-option menu using read + case:

1. System information
2. Disk usage
3. Memory usage
4. Network information
5. Running processes
6. Project directories
7. User and groups
8. Exit

Parts 13–14 — Reports + Permission Verification



Part 13 — system-report.sh

Generate system-report.txt containing date, hostname, user, kernel, uptime, memory, disk, network, users, processes, and directory information.

Use > and >> for redirection.

Part 14 — Simulated admin problem

Temporarily set developers/project.txt to chmod 777, detect the unusual permission, restore it to chmod 660, and verify again.

```
chmod 777 developers/project.txt
./file-audit.sh
chmod 660 developers/project.txt
```

Final System Administration Project — linux-admin.sh



Toolkit menu

1. System information
2. Resource usage
3. Network information
4. Process information
5. File & directory information
6. User information
7. Generate system report
8. Exit

Must use

- Shebang
- Variables
- Command substitution
- for loop
- if / elif / else
- read
- \$1
- case
- Admin commands
- Redirection

Must not use

- Arrays
- Functions
- Bash file tests
- Debugging tools

System Administration — Reflection Questions



Core Bash concepts

- Why is Bash useful for Linux system administrators?
- What is the benefit of automating repeated commands?
- What are variables and command substitution?
- What does \$1 represent?
- Why are loops, if/else, and case useful?

Administration practice

- Why monitor disk, memory, processes, and networking?
- Why create logs or reports?
- What can go wrong with an incorrect automated script?
- Why test scripts before using them on important systems?

Project Directory Structure

Project environment used for the file and directory inspection tasks.



```
permissions-capstone/  
├── public/  
│   └── announcement.txt  
├── private/  
│   └── secrets.txt  
├── developers/  
│   ├── project.txt  
│   └── deploy.sh  
└── shared/
```



Student Submission Checklist

- 1 Create every required .sh script in the correct project location.
- 2 Keep output readable with clear section headings.
- 3 Use only the allowed Bash features and Linux commands.
- 4 Show evidence where a task requests a run or before/after comparison.
- 5 Answer the short reflection questions.
- 6 Verify permissions after temporary chmod changes.
- 7 Generate the required text report files.
- 16 Complete the final integrated toolkit for your course track.

PROJECT GOAL



**Turn manual Linux commands
into repeatable administrative tools.**

Build → run → inspect → verify → report

